

Homework 2

Kristian Kersting, Antonia Wüst, Tim Woydt, Jingyuan Sha, Steven Braun

Submission Guidelines

- Submissions are due on 21.11.25 via Moodle.
- For questions regarding lectures or exercises, please use the Moodle forum.
- If submitting scanned handwritten work (in .pdf format), ensure it is clearly legible. We reserve the right not to grade illegible submissions.
- For programming tasks, only submit the provided skeleton files (hw[X]_ex[Y].py) containing your solutions.

Programming Guidelines

Please adhere to the following for all programming exercises:

- Do not modify the provided class or method signatures in the skeleton files.
- Ensure your submitted code is free of syntax errors and correctly implements the required functionality.
- The use of Large Language Models (e.g., ChatGPT, Copilot) to generate your solutions end-to-end is strictly prohibited and will be treated as plagiarism. Submissions found to be generated by an LLM will receive zero points for the entire homework assignment.

1. Bayesian Networks

(4 Points)

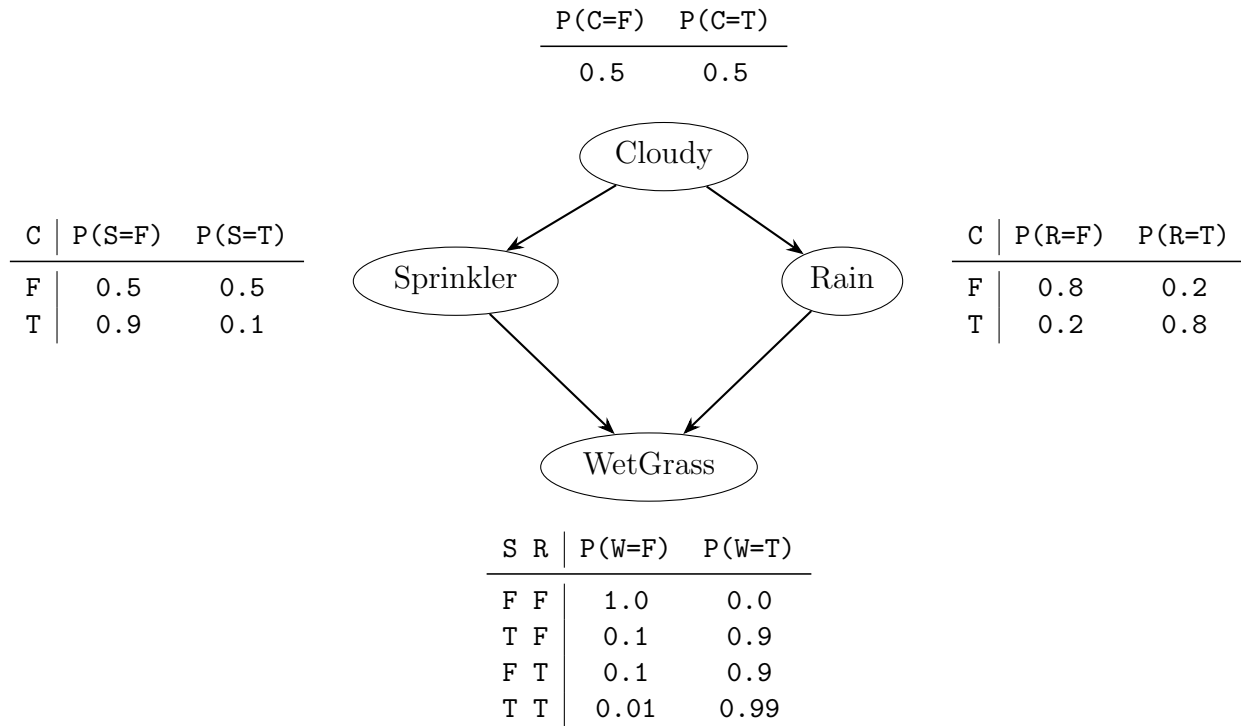


Figure 1: Water Sprinkler Bayesian Network with conditional probability tables.

Consider the water sprinkler Bayes net shown above, where f =false and t =true.

- Provide one factorization for $P(C, S, R, W)$
- Compute $P(S = t|W = t)$
- Compute $P(W = t)$
- Compute $P(S = t|W = t, R = t)$. Is this lower or higher than $P(S = t|W = t)$? Given an intuitive explanation why.

2. Bayesian Networks Programming Exercise

(4 Points)

In this exercise, you will implement a simple Bayesian Network for binary random variables in Python. We provide you with a skeleton file `hw2_ex2.py`. Your task is to fill in your personal details at the top of the file and implement the missing methods. The provided `__main__` block shows an example of how the network is constructed and can be used as a starting point for testing your implementation. Note, that all probability computations should be performed in log-space to ensure numerical stability.

You need to implement the following methods:

- `Node.log_likelihood(self, states)`: This method should compute the conditional log-likelihood of the node being in a specific state, given the states of its parents.
- `BayesianNetwork.joint_log_likelihood(self, states)`: This method calculates the joint log-likelihood of a given complete assignment of states for all nodes in the network. This can be computed using the chain rule for Bayesian networks:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

- `BayesianNetwork.marginal_log_likelihood(self, states)`: This method calculates the probability of a partial assignment. For a query on variables $\mathbf{Q}$, this is computed as:

$$P(\mathbf{Q} = \mathbf{q}) = \sum_{\mathbf{h}} P(\mathbf{Q} = \mathbf{q}, \mathbf{H} = \mathbf{h})$$

where $\mathbf{H}$ are the hidden variables.

- `BayesianNetwork.conditional_log_likelihood(self, states_query, states_evidence)`: This method calculates the conditional probability $P(\text{query} \mid \text{evidence})$. It can be computed using the definition of conditional probability:

$$P(\text{query} \mid \text{evidence}) = \frac{P(\text{query}, \text{evidence})}{P(\text{evidence})}$$

If the evidence probability is zero, we expect the `conditional_probability` method to raise a `ZeroDivisionError`.

3. Conditional Independence

(5 Points)

Consider the following belief network for a medical diagnosis example, where B = Bronchitis, S = Smoker, C = Cough, X = Positive X-ray, L = Lung cancer, and M = Mask. Given the evidence E , see below, which node pairs are conditionally independent?

- There is no evidence for any of the nodes, $E = \emptyset$.
- The Smoke node is given, $E = \{S\}$.
- The Lung cancer node is given, $E = \{L\}$
- Both Lung cancer node and Bronchitis node are given, $E = \{L, B\}$.
- The patient is wearing a mask, $E = \{M\}$

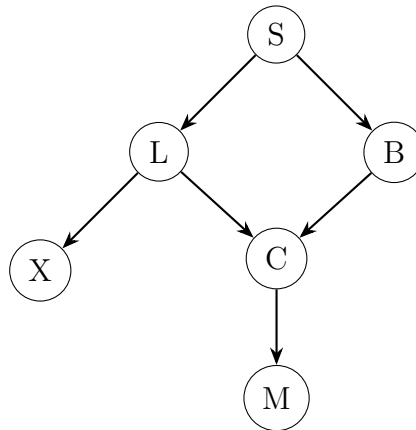


Figure 2: Medical Diagnosis Bayesian Network for disease, symptom, and test relationships.

4. Conditional Independence Programming Exercise (3 Points)

In this exercise, you will implement the d-separation criterion to determine conditional independencies directly from the structure of a Bayesian network. We provide you with a skeleton file `hw2_ex4.py`. Your task is to fill in your personal details at the top of the file and implement the missing methods in the `BayesianNetwork` class. This class focuses only on the graph structure, not the probability distributions.

You need to implement the following methods:

- `BayesianNetwork.is_d_separated(self, x, y, evidence)`: This method should return `True` if node `x` is d-separated from node `y` given the `evidence` set, and `False` otherwise. (2 Points)
- `BayesianNetwork.independencies(self, node, evidence)`: This is the core of the exercise. The method should find all nodes that are d-separated from the input `node` given a set of `evidence` nodes. (1 Point)

5. Markov Blanket (2 Points)

Given a Bayesian network and one of its nodes X , determine the minimum set of evidence nodes which guarantee that X is independent of *every* other node in the network. Prove that this independence holds, for instance using d-separation. This set is also called the Markov blanket, as it “shields” the node from any outside influence.

6. Markov Blanket Programming Exercise (2 Points)

In this exercise, you will implement a method to find the Markov blanket of a node in a Bayesian

Network. We provide you with a skeleton file `hw2_ex6.py`. Your task is to fill in your personal details at the top of the file and implement the missing method `Node.markov_blanket(self)`.

Your implementation of `Node.markov_blanket(self)` should identify the Markov blanket and return it as a single set of `Node` objects. The provided `Node` class structure, which includes lists of parents and children for each node, contains all the information you need to complete this task.